

1. 解决“共享前缀被重复 prefill”的算力浪费。在线服务中大量请求共享同一段前缀：同一 system prompt、同一批 few-shot 示例、多轮对话历史、同一篇 RAG 文档。传统做法让每个请求都从第 0 个 token 重新 prefill，而 prefill 是计算密集型的，开销随前缀长度线性增长，并发越高、前缀越长，浪费越严重。

2. 存储层和调度层

3. 它解决 KV cache 的显存浪费与碎片。PagedAttention 借鉴 OS 虚拟内存分页：把 KV 切成固定大小的 block，逻辑连续的序列经 block table 映射到物理不连续的块，于是实现按需分配（内部碎片只剩最后一页）、无外部碎片、页可被多个序列共享（copy-on-write，支持 beam search 与前缀共享）。联系：① 同一套抽象——都放弃“一个请求一块连续 KV”，改用固定页 + 间接映射，本质是 OS 分页思想在 KV cache 上的移植；② 同一目标——提高显存利用率 → 抬高可行的 batch size → 提升吞吐；③ 同一粒度——都以“块/页”作为共享与复用的最小单位，也都用哈希判断“这段 token 是否已有缓存”，SGLang 底层同样是页式 KV 池。差异：① 索引结构不同——vLLM 的前缀缓存（APC）用哈希表加块哈希，回答的是“这个块在不在”，本质是按块查表；SGLang 的 RadixCache 是一棵基数树，回答的是“最长公共前缀有多长”，天然支持任意深度的部分匹配与分叉（同一 system prompt 下挂不同用户问题）。② 默认行为不同——RadixAttention 是 SGLang 的设计中心，前缀复用默认开启、自动生效；PagedAttention 本身只管“怎么存”，前缀复用是叠在它之上、需要显式开启的可选能力。③ 定位不同——PagedAttention 是一套内存管理机制；RadixAttention 是内存管理 + 调度策略的组合。SGLang 可以理解为：底层存储与 vLLM 同为页式，上层用基数树统一做全局索引与调度。